LAB 5

Next.js + FastAPI: CORS and the API Contract

Goal

Connect two projects that have lived separately until now: the course catalog on Next.js with data from lib/courses.ts (Lab 1) and the FastAPI backend with the same six courses (Lab 4). By the end of today's work, the Next.js pages get their courses from FastAPI, and the browser itself sends likes and course enrollments to FastAPI.

Both projects change today. The backend gains CORS, camelCase JSON, and two routes that accept data. The frontend gains a .env.local file, real requests instead of mock functions, a Client Component with SWR, a working like button, and an enrollment form. The second half of the lab is diagnostics: you'll deliberately break CORS in several ways and find the cause of each breakage using the algorithm from section 7 of Lecture 5, not by trial and error.

Before you start

- both projects in working order: course-catalog (Lab 1, with the styles from Lab 3 if you have them) and course-catalog-api (Lab 4);
- Node.js 20 or newer and Python 3.10 or newer — the same as in Labs 1 and 4;
- a Chromium-based browser (Chrome, Edge, Brave): the error messages below are quoted for it;
- Lecture 5's handout open nearby — sections 4–7 (CORS, the API contract, environment variables, the algorithm for finding the cause): today you'll repeat its examples almost word for word;
- two terminal windows: one runs the backend (with .venv active), the other the frontend; a third will come in handy for git and curl.

If you're opening DevTools for the first time

Almost all of today's work happens in the browser's DevTools. The minimum you'll need:

- open DevTools with F12 (on macOS — Cmd+Option+I) on your application's page;
- the Console tab — this is where the browser prints errors, CORS ones included; read the red messages in full, from the first word;
- the Network tab — a table of all the page's requests. Open it before the action you're studying: requests sent earlier won't appear in the table (in that case, reload the page with the tab open). In the filter, choose Fetch/XHR so you don't see images and scripts;
- clicking a request row opens its details. On the Headers tab: the address (Request URL), the method (Request Method), the status (Status Code), the request headers (Request Headers), and the response headers (Response Headers). On the Response tab: the response body;
- the Disable cache checkbox (Network tab) turns off the browser cache; you'll need it in section 9 for the preflight;
- a request the browser blocked is marked red, and its Status column says CORS error (in older versions, failed). That's not the same as a response with a 4xx or 5xx code.

What you'll build

A chain of two applications, with requests travelling along two different paths (Diagram A in Lecture 5): requests from a Server Component are made by the Next.js server, requests from a Client Component by the browser itself. The five request paths are in the table below.

What	Who sends the request	FastAPI route	CORS
The /courses page	the Next.js server (Server Component)	GET /courses	not needed
The /courses/[id] page	the Next.js server (Server Component)	GET /courses/{course_id}	not needed
The “Electives only” filter	the browser (Client Component, SWR)	GET /courses?is_elective=true	needed
The like button	the browser (Client Component)	POST /courses/{course_id}/like	needed
The enrollment form	the browser (Client Component)	POST /enrollments, JSON body	needed + preflight

The structure of the two projects by the end of the lab (changes are marked):

```
project structure
course-catalog-api/                # backend (Lab 4)
├── main.py                        # + CORS, + like and enrollments routes
├── models.py                      # + CamelModel, + enrollment models
└── data.py                        # unchanged

course-catalog/                    # frontend (Labs 1–3)
├── .env.local                     # new: the API address
├── next.config.ts                 # check: no cacheComponents
├── lib/courses.ts                 # getCourses and getCourse call the API
├── components/
│   ├── ElectiveFilter.tsx         # new: SWR from the browser
│   ├── LikeButton.tsx             # the like goes to the server
│   └── EnrollForm.tsx             # new: the enrollment form
└── app/
    ├── layout.tsx                 # + a "Filter" link
    ├── filter/page.tsx            # new
    └── courses/
        ├── error.tsx              # new (Lab 1 bonus task)
        └── [id]/page.tsx          # no generateStaticParams
```

1. Starting Check: Each Half on Its Own

Before connecting the applications, make sure each one works by itself. If something breaks after the connection, you'll know the cause is in today's code, not in the old one.

Backend. In the first terminal window, go to the course-catalog-api folder, activate the environment, and start the server (the same commands as in Lab 4):

```
terminal — window 1 (backend)
cd course-catalog-api
source .venv/bin/activate        # Windows: .venv\Scripts\Activate.ps1
fastapi dev main.py
```

Open http://localhost:8000/docs — there should be the three routes from Lab 4. Then open http://localhost:8000/courses and note what the elective field is called: right now it's is_elective. In a few minutes that will change.

Frontend. In the second terminal window, start the Next.js dev server:

```
terminal — window 2 (frontend)
cd course-catalog
npm run dev
```

Open http://localhost:3000/courses. For now this is mock data from lib/courses.ts: six cards in array order (ai-integration last) and a 300 ms delay.

> **Check next.config.ts:** the create-next-app 16.4 wizard asks about Cache Components, and if you accept the default answer (Yes), two lines end up in the config: cacheComponents: true and partialPrefetching: true. That is a different caching model, while the pages from Labs 1–5 are written for the classic one: in the new mode npm run build fails on /courses with an error about accessing data outside a <Suspense> boundary. If you have these lines, delete both (leave the turbopack block and everything else as it is). If you don't, there's nothing to do.

Git. Run git status in both projects. The working tree should be clean (nothing to commit, working tree clean). If there are leftover changes, commit them now so that today's changes are easy to tell apart from the old ones.

> **The frontend address is http://localhost:3000, and only that.** That exact origin is the one you'll allow in the backend today. If port 3000 is taken and Next.js picked 3001, stop the extra process that occupies port 3000 and start the dev server again: the backend won't allow an origin with a different port (you'll see this with your own eyes in section 10).

> **Check yourself:** /docs shows three routes; in the frontend, /courses shows six courses; there's no cacheComponents in next.config.ts; git status is clean in both projects. If even one item doesn't hold, fix it now, not after connecting.

2. Backend: Adding CORS

We start with the backend. First, let's see with our own eyes that, without CORS configured, FastAPI gives the browser no permission at all. Request the root route the way a page from localhost:3000 would — that is, with an Origin header:

```
terminal — window 3
curl -i -H "Origin: http://localhost:3000" http://localhost:8000/
```

> **Windows PowerShell:** in some versions the word curl is an alias for a different command (Invoke-WebRequest), and the -i and -H flags won't work. Type curl.exe — that's the real curl, built into Windows 10 and later. In the cmd command prompt and on macOS/Linux, plain curl works.

The command prints the response headers (-i) and the body. The expected result, before CORS is connected:

```
expected output — CORS not connected yet
HTTP/1.1 200 OK
date: Sat, 10 Oct 2026 14:20:36 GMT
server: uvicorn
content-length: 43
content-type: application/json

{"message":"Course Catalog API is running"}
```

The server answered 200 and executed the request, but there are no access-control-* headers in the response. A browser that received such a response to a request from a page on another origin would hide it from JavaScript. Now grant the permission. In main.py, add the CORSMiddleware import and attach it to the application right after app = FastAPI(...):

```
main.py — skeleton
from fastapi.middleware.cors import CORSMiddleware   # new import

app = FastAPI(title="Course Catalog API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[],   # TODO: the frontend origin (scheme, host, port)
    allow_methods=[],   # TODO: allow all methods
    allow_headers=[],   # TODO: allow all headers
)
```

> **allow_origins is a list of origins, not page addresses:** scheme, host, and port, with no path and no trailing slash. Breaking this rule is the cause of most failed CORS setups (section 4 of the lecture); in section 10 you'll reproduce it on purpose.

Save the file: fastapi dev restarts by itself. Repeat the request with Origin and add a request with a foreign origin:

```
terminal — window 3
curl -i -H "Origin: http://localhost:3000" http://localhost:8000/
curl -i -H "Origin: http://evil.example" http://localhost:8000/
```

Request	Expected result
Origin: http://localhost:3000	the response now contains the lines access-control-allow-origin: http://localhost:3000 and vary: Origin
Origin: http://evil.example	the status is still 200, but there's no access-control-allow-origin line: this origin hasn't been given permission

> **Note:** for the request with a foreign origin the server also answered 200 and executed it. CORS forbids the server nothing — it only decides whether the browser gets permission to read the response. CORS is not a way to protect data (section 2 of the lecture).

> **Check yourself:** the first request returned access-control-allow-origin with the value http://localhost:3000 — character for character, with no trailing slash; the second one doesn't contain such a line. If the first response has no such header, open main.py and compare allow_origins with the task.

3. Backend: camelCase Names in JSON

Now the contract of names. Open http://localhost:8000/courses and find the elective field: the backend sends is_elective, while the TypeScript type Course from Lab 1 expects isElective. If you leave everything as it is, the page won't show a single error: for every course, course.isElective will be undefined, that is, “not an elective” (section 5 of the lecture). We chose approach A: Python keeps snake_case, and Pydantic itself translates the names in JSON.

First, check the Pydantic version: the validate_by_name setting you'll need below appeared in version 2.11.

```
terminal — window 3 (with .venv active)
pip show pydantic
```

The Version line should say 2.11 or higher. If the version is lower, update the packages: pip install -U "fastapi[standard]". In models.py, create a common base class and inherit Course from it:

```
models.py — skeleton
from pydantic import BaseModel, ConfigDict          # add ConfigDict
from pydantic.alias_generators import to_camel      # new import

class CamelModel(BaseModel):
    model_config = ConfigDict(
        # TODO 1: alias_generator=to_camel — aliases in camelCase
        # TODO 2: validate_by_name=True — also accept the field name
        # TODO 3: validate_by_alias=True — and the alias
    )

class Course(BaseModel):   # TODO 4: inherit from CamelModel
    id: str
    title: str
    description: str
    credits: int
    is_elective: bool = False   # in JSON this will be isElective
    likes: int = 0
```

Save and open /courses again. Expected result: the keys are in camelCase, and ai-integration comes first:

```
the first item of the /courses response (formatted for reading)
{
  "id": "ai-integration",
  "title": "AI/LLM Integration",
  "description": "LLM features in an app via the OpenAI API.",
  "credits": 5,
  "isElective": true,
  "likes": 32
}
```

Open /docs and expand the Schemas section: in the Course model the field is also called isElective. You didn't have to change the routes: by default FastAPI writes fields under their aliases.

A trap from the lecture — see it with your own eyes

Temporarily comment out the validate_by_name=True line and save the file. Open /courses: all six courses have isElective equal to false, and the request /courses?is_elective=true returns an empty list []. No error in the terminal, no warning in the browser — electives silently stopped being electives. The reason: without validate_by_name, Pydantic accepts only aliases when creating an object, while in data.py the courses are created by field name (is_elective=True); the extra argument is quietly ignored. Put the line back and make sure /courses?is_elective=true returns two courses again.

Request	Expected result
/courses	six courses, camelCase keys: id, title, description, credits, isElective, likes
/courses?is_elective=true	2 courses: ai-integration and api-design
/courses?isElective=true	all 6 courses: there's no parameter with that name, and FastAPI silently ignores an unknown parameter

Parameter names don't change: aliases apply to the request and response body, but not to query parameters — those are declared in the function signature and stay exactly as you named them (section 5 of the lecture). So the frontend filter will request ?is_elective=true, not ?isElective=true.

Check yourself: the table above matched in full, and the requests from the Lab 4 tables still work the same way: for example, ?sort=title and ?page=2&page_size=2.

4. Backend: Likes and Course Enrollment

So far the backend only served data (GET). Today it accepts data for the first time: a like increments a course's counter, and an enrollment receives JSON from the client. Both routes change data in the server's memory: you'll connect a database in weeks 6–7, so after every restart of fastapi dev the like counters and the list of enrollments return to their initial state.

Like: POST /courses/{course_id}/like

The route is similar to GET /courses/{course_id}: it looks up a course and answers 404 if there isn't one. The differences are the method and a side effect. It increments the counter and returns the updated course, so that the frontend can show the current number without a second request.

```
main.py — route skeleton
@app.post("/courses/{course_id}/like", response_model=Course)
def like_course(course_id: str):
    course = find_course(course_id)
    # TODO 1: if not found — 404 "Course not found" (like get_course)
    # TODO 2: increase course.likes by 1
    return course
```

New: @app.post. The decorator works like @app.get, but for the POST method. POST is used for actions that change the server's state, so the route can't be checked by simply opening its address in the browser: the address bar always sends GET. We'll check it through /docs.

Course enrollment: POST /enrollments

The second route accepts data: the course id and the student's name in the request body. The request body is described by a Pydantic model — the same technique as for the response (Lecture 4), only now the model sits in the function's parameter. Add two models to models.py; copy them as they are, because Field wasn't covered in the lecture:

```
models.py — changes
# at the top of the file, add Field to the existing import:
from pydantic import BaseModel, ConfigDict, Field

# at the end of the file, add two models:
class EnrollmentIn(CamelModel):
    course_id: str
    student_name: str = Field(min_length=2)

class Enrollment(EnrollmentIn):
    id: int
```

EnrollmentIn describes what the client sends, and Enrollment inherits those fields and adds an id, which the server assigns. Both models inherit from CamelModel, so in JSON the fields are called courseId and studentName. Now the route itself:

```
main.py — route skeleton
from models import Course, Enrollment, EnrollmentIn   # at the top: 2 new models

ENROLLMENTS: list[Enrollment] = []   # an in-memory "database" on the server

@app.post("/enrollments", response_model=Enrollment, status_code=201)
def create_enrollment(data: EnrollmentIn):
    # TODO 1: if there's no course data.course_id (find_course returned
    #         None) — 404 "Course not found", as in get_course
    # TODO 2: create an Enrollment: id = len(ENROLLMENTS) + 1,
    #         take course_id and student_name from data
    # TODO 3: append the record to ENROLLMENTS
    return {}  # TODO 4: return the created record
```

For checking, add one more route — it will show that the enrollments really reached the server. Copy it in full:

```
main.py — at the end of the file
@app.get("/enrollments", response_model=list[Enrollment])
def list_enrollments():
    return ENROLLMENTS
```

This route has three novelties:

What	What it does
data: EnrollmentIn	FastAPI treats a parameter whose type is a model as the request body: it reads the JSON, validates it against the model, and itself answers 422 on an error — the function receives data that has already been validated
status_code=201	code 201 Created is the convention for a POST that creates a new record; by default FastAPI would have answered 200
Field(min_length=2)	an extra constraint on the field: a name shorter than two characters is rejected with code 422, and the response body explains exactly what's wrong

Checking in /docs

/docs now shows six routes. For each POST, expand the route, click Try it out, fill in the fields, and click Execute. Compare with the table:

What you do	Expected result
POST /courses/web-security/like	200; the body is the web-security course, likes went up by 1 (with the original data it was 21 and is now 22)
the same again	200, likes: 23
POST /courses/nope/like	404, body {"detail": "Course not found"}
POST /enrollments, body {"courseId": "web-security", "studentName": "Aigerim"}	201; body {"courseId": "web-security", "studentName": "Aigerim", "id": 1}
the same, but "studentName": "A"	422; in detail: "loc": ["body", "studentName"], "msg": "String should have at least 2 characters"
the same, but "courseId": "nope"	404, Course not found
GET /enrollments	a list with one record (Aigerim)

> **Note loc:** in the 422 error the field is named studentName, not student_name: error messages use the same name under which the field is sent in JSON.

> **Check yourself:** /docs shows six routes, the table above matched in full, and after the server restarts (save main.py) the likes and enrollments are reset again — that's how it should be. The backend is now ready: make a commit in course-catalog-api.

5. Frontend: the API Address and Real Requests

The API address in .env.local

The backend address isn't hard-coded: on your computer it's http://localhost:8000, and after deployment it will be something quite different. In Next.js such values live in a .env.local file. Create it in the root of course-catalog, next to package.json:

```
.env.local
API_URL=http://localhost:8000
NEXT_PUBLIC_API_URL=http://localhost:8000
```

The two lines are two different kinds of variables (section 6 of the lecture). A variable without a prefix is visible only to code that runs on the Next.js server: Server Components and Route Handlers. A variable with the NEXT_PUBLIC_ prefix is written into the JavaScript that the browser downloads when the project is built, so it's available in a Client Component — and that's why you must never put anything secret in it. Right now you need the first one (for getCourses and getCourse); the second will be needed in sections 8 and 9.

Restart the dev server (Ctrl+C, then npm run dev again): the beginning of the output should now contain a line like - Environments: .env.local. From then on, Next.js re-reads the file itself when it changes (you'll see Reload env: .env.local in the terminal).

> **The .env.local file must not end up in git.** A project created with create-next-app already includes .env* in .gitignore: make sure git status doesn't show .env.local. Since the file won't be in the repository, describe in the README which variables the project needs (names and an example value) — otherwise whoever checks your work won't be able to run it.

lib/courses.ts: the functions stay, their bodies change

In Lecture 4 we promised that getCourses() and getCourse() would stay in place and the pages wouldn't notice. Open lib/courses.ts. Leave the Course type as it is, add an Enrollment type (you'll need it in section 9), and delete the courses array and the delay function — the data now lives in the backend. Then rewrite both functions:

```
lib/courses.ts — skeleton
export type Course = {
  // unchanged (Lab 1)
};

export type Enrollment = {
  id: number;
  courseId: string;
  studentName: string;
};

const API_URL = process.env.API_URL;
if (!API_URL) throw new Error("API_URL is not set: create .env.local");

export async function getCourses(): Promise<Course[]> {
  // TODO 1: fetch(`${API_URL}/courses`, ...) with the option cache: "no-store"
  // TODO 2: if !res.ok — throw Error(`API error: ${res.status}`)
  // TODO 3: return res.json()
}

export async function getCourse(id: string): Promise<Course | undefined> {
  // the same for `${API_URL}/courses/${id}`, but before checking res.ok:
  // if res.status === 404, that is not an error but "no such course":
  // return undefined
}
```

- cache: "no-store" tells Next.js to request the data on every visit to the page. Without it, the page may be built in advance and the data will “freeze” — the trap from section 6 of the lecture; you'll see its result in the build table in section 6;
- fetch doesn't throw an error on 404 and 500 responses: to it, those are “successful” responses, so res.ok is checked by hand and the error is thrown by the code itself;
- for getCourse, status 404 is a normal result (“no such course”), and the page turns it into notFound(). Any other unsuccessful status is a real error, and error.tsx will catch it;

- the if (!API_URL) check at the top of the module gives a quick error with clear text instead of requests to an address like undefined/courses.

The pages don't change: app/courses/page.tsx still calls await getCourses(). Check the result (both servers running):

Address	Expected result
/courses	six cards, but the order has changed: ai-integration first, api-design last (with the original data they have 32 and 11 likes). That's how the backend sorts (sort=popular is the default); the 300 ms delay is gone
/courses/web-security	the course page: title, description, 4 credits, and a ❤️ button with the like count from the backend (the button still works locally, as in Lab 1)
/courses/does-not-exist	your not-found.tsx page

Prove that the request comes from the Next.js server

Open /courses and DevTools → Network → Fetch/XHR, then reload the page. The table has no requests to localhost:8000: the HTML came from Next.js with the courses already inside. And in the backend's terminal window, log lines have appeared that the Next.js process left when it asked for the data:

```
backend terminal (window 1)
INFO:     127.0.0.1:52304 - "GET /courses HTTP/1.1" 200 OK
INFO:     127.0.0.1:52306 - "GET /courses/web-security HTTP/1.1" 200 OK
INFO:     127.0.0.1:52310 - "GET /courses/does-not-exist HTTP/1.1" 404 Not Found
```

The numbers after 127.0.0.1: will be different for you. What matters is something else: this is path 1 from Diagram A — “server to server.” There's no browser in the chain, so browser rules don't apply to this request and it doesn't need CORS. The /courses page would work even if you hadn't connected CORSMiddleware — you'll verify this in section 10.

> **Check yourself:** the order of the cards has changed (ai-integration first), Network shows no requests to localhost:8000, and the backend terminal shows GET requests. If the page shows an error, read its text: most often it's API URL is not set (.env.local wasn't created or the dev server wasn't restarted), or the backend isn't running. Make a commit in the frontend.

6. The Build: What Happens During next build

The dev server forgives a lot: pages are built on request, and if the backend is off, you simply see an error. A production build works differently: Next.js builds some pages in advance, when you run npm run build — that is, it requests data before anyone has opened the site. Let's see what that means for your project.

Stop the backend (Ctrl+C in window 1); you can leave the frontend dev server running. In the third terminal window, in the course-catalog folder, start the build:

```
terminal — window 3 (frontend)
npm run build
```

If app/courses/[id]/page.tsx still has generateStaticParams from Lab 1, the build will fail. Expected output (abridged; your line numbers will differ):

```
expected output — the build fails
  Collecting page data using 1 worker ...
TypeError: fetch failed
    at async c (lib/courses.ts:20:15)
    at async g (app/courses/[id]/page.tsx:7:19)
  ...
  [cause]: Error: connect ECONNREFUSED 127.0.0.1:8000
  ...
> Build error occurred
Error: Failed to collect page data for /courses/[id]
```

The cause isn't a page but the generateStaticParams function. It asks Next.js to build the course pages in advance and for that calls getCourses() right during the build. The backend is off, the connection is refused (ECONNREFUSED), and the build stops. The lecture (section 6) describes a similar trap for pages without cache: "no-store"; here a second source of build-time requests triggered it. If your build passed, then generateStaticParams is already gone from your project — move on to the route table below.

There are two ways out: keep the backend running for every build, or give up building the course pages in advance. The list of courses now belongs to the backend and changes without rebuilding the frontend, while a course page built in advance would show stale data, so we choose the second. Delete the generateStaticParams function from app/courses/[id]/page.tsx along with the getCourses import, which is no longer needed, and run npm run build again with the backend off. Now the build passes:

```
expected output — the build passes (excerpt)
Route (app)
┌ ○ /
├ ○ /_not-found
├ ○ /about
├ ƒ /courses
└ ƒ /courses/[id]

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

Read the table. The ○ symbol means a page built in advance — opening it doesn't touch the backend. The *ƒ* symbol means a page that is built on every request. /courses and /courses/[id] are marked *ƒ*: it's cache: "no-store" that makes a page dynamic, because its data has to be fresh. If /courses is marked ○ for you, then cache: "no-store" is missing in getCourses and the page is frozen on the data from the moment of the build (in bonus task 4 you'll see this with your own eyes). Bring the backend back: fastapi dev main.py.

> **In dev mode the same cause looks different.** If you keep generateStaticParams and turn the backend off, the course page shows a blank screen with a 500 error, not your error.tsx: the call to generateStaticParams happens outside the error boundary. That's one more reason to get rid of it.

> **Check yourself:** npm run build passes with the backend off; /courses and /courses/[id] are marked *ƒ*; the course's page.tsx has no generateStaticParams. The Lab 1 criterion about generateStaticParams still counts as met: you implemented it when the data was a mock function and deliberately removed it when the data moved to the backend. Explain this decision in the README in a sentence or two.

7. loading, not-found, and error with a Real Backend

In Lab 1 you prepared three special App Router files. Back then the data came from mock functions, and all three situations — waiting, “no such course,” an error — had to be imitated. Now each has a real cause. Check them in turn:

File	When it triggers now	How to trigger it
loading.tsx	while the Server Component waits for the backend's response	slow the backend down (see below)
not-found.tsx	the backend answered 404, getCourse returned undefined, and the page called notFound()	open /courses/does-not-exist
error.tsx	the backend is unavailable or answered with an error, and getCourses threw an exception	stop the backend and open /courses

loading.tsx: making the backend slow

Before, the loading screen was visible thanks to the 300 ms delay inside the mock functions. Now the request is real and takes milliseconds, so loading.tsx is barely noticeable. Let's imitate a slow backend: add a delay to get_course in main.py.

```
main.py — a temporary change
import time                        # at the top of the file

@app.get("/courses/{course_id}", response_model=Course)
def get_course(course_id: str):
    time.sleep(1)                  # temporary: imitates a slow backend
    ...                            # the rest of the function is unchanged
```

Open /courses and click any card: your loading.tsx will appear for a second (for example, “Loading the course…”). It's visible both when you navigate via a link and when you open the address directly. Then delete time.sleep(1) and import time.

not-found.tsx: no such course

Open /courses/does-not-exist. The chain is as follows: Next.js sends GET /courses/does-not-exist, FastAPI answers 404, getCourse sees res.status === 404 and returns undefined, the page calls notFound(), and your not-found.tsx is shown. In the backend terminal there's a line with 404 Not Found (see section 5). Note that for getCourse, 404 is a normal result, not an error. If the function threw an exception on any unsuccessful status, you'd see error.tsx instead of “Course not found.”

error.tsx: stopping the backend

If you don't have app/courses/error.tsx (in Lab 1 it was an optional task), create it. It must be a Client Component ("use client"): the file receives the error and a retry function. Copy it in full; the styling can be replaced with your own:

app/courses/error.tsx

```
"use client";

import { useEffect } from "react";

export default function Error({
  error,
  retry,
}: {
  error: Error & { digest?: string };
  retry: () => void;
}) {
  useEffect(() => {
    console.error(error);
  }, [error]);

  return (
    <main className="p-6">
      <h1 className="mb-2 text-2xl font-bold">Something went wrong</h1>
      <p className="mb-4">
        Could not load the data. The backend may not be running.
      </p>
      <button
        onClick={() => retry()}
        className="rounded-md border border-slate-300 px-3 py-1"
      >
        Try again
      </button>
    </main>
  );
}
```

> **New: retry, not reset.** In Next.js 16.3 and newer, error.tsx receives a retry function: it requests the data from the server again and re-renders the segment. The familiar reset function only re-renders the component and doesn't reload the data — after the backend goes down, it won't help. In Next.js 16.2 the same capability was called unstable_retry. The npm ls next command shows your version.

Stop the backend (Ctrl+C) and open /courses. Instead of the cards, your error.tsx appears. The browser console will have a TypeError: fetch failed entry, and in the corner of the page there's a small red “1 Issue” indicator: that's how the dev server reports an error that error.tsx caught. Start the backend again and click “Try again”: the list comes back without a page reload. The same file catches errors on /courses/web-security too: error.tsx applies to the whole courses segment and everything nested in it.

> **Check yourself:** with time.sleep(1) you see loading.tsx; /courses/does-not-exist shows not-found, and the backend terminal shows a 404 Not Found line; with the backend stopped, /courses and /courses/web-security show error.tsx, and the button brings the page back once the backend is running. Don't forget to remove time.sleep(1). Make a commit.

8. A Request from the Browser: the “Electives only” Filter

Until now, every request to the backend was sent by the Next.js server. Now we'll add the first request that the browser itself sends — path 2 from Diagram A. This is where CORS comes in, the reason you configured the backend in section 2. The idea is the same as in Lecture 2: a checkbox above the list, and the data is re-requested without reloading the page — but the address is now absolute.

Install SWR (in the course-catalog folder) if it isn't installed yet:

```
terminal — frontend
npm install swr
```

Create components/ElectiveFilter.tsx. The first part is the API address and the fetcher:

```
components/ElectiveFilter.tsx — part 1: fetcher
"use client";

import { useState } from "react";
import useSWR from "swr";
import type { Course } from "@/lib/courses";

const API = process.env.NEXT_PUBLIC_API_URL;

async function fetcher(url: string): Promise<Course[]> {
  const res = await fetch(url);
  // TODO: if !res.ok — throw Error(`API error: ${res.status}`)
  return res.json();
}
```

- import type means that only the type is taken from lib/courses.ts. The module itself is meant for the server (it contains the API_URL check) and must not end up in browser JavaScript;
- the fetcher checks res.ok: fetch rejects its promise only on a network failure, and a 404 or 500 response is a “success” to it. Without the check, the error body {"detail": …} would end up in data and data.map would crash. SWR counts as an error only what the fetcher throws, so we throw it ourselves.

The second part is the component itself (a continuation of the same file):

components/ElectiveFilter.tsx — part 2: the component

```
export function ElectiveFilter() {
  const [onlyElective, setOnlyElective] = useState(false);
  const query = ""; // TODO 1: "?is_elective=true" if the checkbox is on
  const { data, error, isLoading } = useSWR(`${API}/courses${query}`, fetcher);

  return (
    <div>
      <label className="flex items-center gap-2 mb-4">
        <input
          type="checkbox"
          checked={onlyElective}
          onChange={(e) => setOnlyElective(e.target.checked)}
        />
        Electives only
      </label>
      {error && <p>Failed to load courses</p>}
      {isLoading && <p>Loading…</p>}
      {data && (
        <ul className="list-disc pl-6">
          {data.map((c) => (
            <li key={c.id}>
              {/* TODO 2: the title, " — N cr." and, if it is an elective, */}
              {/* " · elective" (hint: c.isElective && "…") */}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

The query parameter is still called is_elective: aliases apply only to the request and response body (section 3). All that's left is to hook up the component: create the page app/filter/page.tsx and add a “Filter” link to the navigation in app/layout.tsx, following the example of the neighboring links.

app/filter/page.tsx

```
import { ElectiveFilter } from "@/components/ElectiveFilter";

export default function FilterPage() {
  return (
    <main className="p-6">
      <h1 className="text-2xl font-bold mb-4">Course filter</h1>
      <ElectiveFilter />
    </main>
  );
}
```

app/layout.tsx — next to the other navigation links

```
<Link href="/filter" className="px-3 py-2 rounded-md hover:bg-slate-100">
  Filter
</Link>
```

Check: what happens in Network

Open http://localhost:3000/filter, then DevTools → Network → Fetch/XHR, and reload the page. The table should have one courses request — this time the browser sent it. Open it and find on the Headers tab:

- Request URL: http://localhost:8000/courses — an absolute address on another origin;
- in Request Headers: Origin: http://localhost:3000 — you didn't write this header, the browser added it itself;
- in Response Headers: access-control-allow-origin: http://localhost:3000 — the permission you configured in section 2. It's what allowed JavaScript to read the response.

Turn on the “Electives only” checkbox: a new request, courses?is_elective=true, appears in the table, and the list keeps two courses marked “elective” (ai-integration and api-design). Turn the checkbox off — six again.

> **If the page keeps showing “Loading…” or “Failed to load courses”:** open Console and Network. A red CORS message means section 2 wasn't done exactly (check allow_origins); a request to /undefined/courses means .env.local wasn't created or the dev server wasn't restarted (section 5). You'll do a detailed analysis in section 10.

> **Check yourself:** /filter shows six courses, and after the checkbox, two marked “elective”; Network shows a request to localhost:8000 with an Origin header in the request and access-control-allow-origin in the response. Make a commit.

9. Like and Enrollment Form: POST from the Browser

Two user actions remain. Both requests are sent by the browser, both go to another origin — and there is a fundamental difference between them that you'll see on the Network tab.

The like button

The LikeButton from Lab 1 counted likes only in the page's memory: after a reload the number went back to the original. Now a click will send a POST to the backend, and the component will take the new number from

the server's response. For that, the component needs a second prop — courseId; pass it on the course page: <LikeButton courseId={course.id} initialLikes={course.likes} />.

components/LikeButton.tsx — skeleton

```
"use client";

import { useState } from "react";
import type { Course } from "@/lib/courses";

const API = process.env.NEXT_PUBLIC_API_URL;

type LikeButtonProps = {
  courseId: string;   // new prop
  initialLikes: number;
};

export function LikeButton({ courseId, initialLikes }: LikeButtonProps) {
  const [likes, setLikes] = useState<number>(initialLikes);
  const [failed, setFailed] = useState(false);

  async function handleClick() {
    setFailed(false);
    try {
      // TODO 1: fetch with method: "POST" to `${API}/courses/${courseId}/like`
      // TODO 2: if !res.ok — throw Error
      // TODO 3: read the course from the response (type Course) via res.json()
      // TODO 4: setLikes(course.likes) — the server's number, not likes + 1
    } catch {
      setFailed(true);
    }
  }

  return (
    <div>
      <button
        onClick={handleClick}
        className="rounded-md border border-slate-300 px-3 py-1"
      >
        ❤️ {likes}
      </button>
      {failed && <p className="text-red-600">Failed to like the course</p>}
    </div>
  );
}
```

If in Lab 3 you replaced the button with the Button component from shadcn/ui, keep it: styling doesn't affect the logic. Check:

- open /courses/web-security and note the number next to ❤️ (after a backend restart it's 21). ClickS ❤️: the number goes up by 1. Reload the page — the new number was saved (in Lab 1 it went back to the original after a reload);
- on the Network tab, a click produces exactly one like request with status 200. There's no OPTIONS before it: a POST with no body and no special headers counts as a simple request, and it needs no preflight;
- open /courses: the number on the card has updated too, because the server builds the page with cache: "no-store". After several likes the order of the cards may change — the backend sorts by popularity.

The enrollment form

Now a request with a JSON body. Create components/EnrollForm.tsx and hook it up on the course page under the like button: <EnrollForm courseId={course.id} />, with the heading “Enroll in this course.” The first part is the sending logic:

components/EnrollForm.tsx — part 1: logic

```
"use client";

import { useState, type FormEvent } from "react";
import type { Enrollment } from "@/lib/courses";

const API = process.env.NEXT_PUBLIC_API_URL;

type EnrollFormProps = {
  courseId: string;
};

type Status = "idle" | "sending" | "done" | "rejected" | "error";

export function EnrollForm({ courseId }: EnrollFormProps) {
  const [studentName, setStudentName] = useState("");
  const [status, setStatus] = useState<Status>("idle");
  const [message, setMessage] = useState("");

  async function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setStatus("sending");
    try {
      const res = await fetch(`${API}/enrollments`, {
        method: "POST",
        // TODO 1: a Content-Type header with the value "application/json"
        // TODO 2: body — JSON.stringify({ courseId, studentName })
      });
      if (res.status === 422) {
        // TODO 3: the server rejected the data. The response body is
        //   { detail: [{ loc, msg, type }, ...] }
        //   save detail[0].msg in message, status "rejected"
        return;
      }
      if (!res.ok) throw new Error(`API error: ${res.status}`);
      const enrollment: Enrollment = await res.json();
      setMessage(`Enrollment number: ${enrollment.id}`);
      setStatus("done");
    } catch {
      setStatus("error");
    }
  }
```

The second part is the markup. Copy it right after handleSubmit, unchanged:

```
components/EnrollForm.tsx — part 2: markup
  return (
    <form onSubmit={handleSubmit} className="flex max-w-sm flex-col gap-2">
      <label htmlFor="studentName">Your name</label>
      <input
        id="studentName"
        value={studentName}
        onChange={(e) => setStudentName(e.target.value)}
        className="rounded-md border border-slate-300 px-3 py-2"
      />
      <button
        type="submit"
        disabled={status === "sending"}
        className="rounded-md border border-slate-300 px-3 py-2"
      >
        Enroll
      </button>
      {status === "done" && <p>You are enrolled. {message}</p>}
      {status === "rejected" && (
        <p className="text-red-600">The server rejected the data: {message}</p>
      )}
      {status === "error" && (
        <p className="text-red-600">Could not submit the enrollment</p>
      )}
    </form>
  );
}
```

> **Name-length validation on the client is deliberately not added.** Today you need to see how the backend rejects data and how the frontend shows that refusal: status 422 and the detail array from section 4. In a real project you check in both places: on the client for the user's convenience, and on the server for reliability, because a client-side check is easy to bypass.s

Preflight on the Network tab

Open a course page and DevTools → Network → Fetch/XHR, clear the table, and go through the steps in order:

- submit the form with a one-letter name, for example A. Under the form you'll see “The server rejected the data: String should have at least 2 characters”. The table has two enrollments rows. The first is the preflight: the method is OPTIONS, the Type column says preflight, status 200. The second is the POST itself with status 422;
- open the preflight row. In Request Headers find Origin: http://localhost:3000, Access-Control-Request-Method: POST, and Access-Control-Request-Headers: content-type — that's the browser's question, “is this OK?” In Response Headers find access-control-allow-origin, access-control-allow-methods, access-control-allow-headers, and access-control-max-age: 600 — the server's answer;
- submit the form again with a proper name (Aigerim). Under the form you'll see “You are enrolled. Enrollment number: 1”. The table shows only a POST with status 201, no OPTIONS: the browser remembered the permission for 600 seconds;
- turn on the Disable cache checkbox and submit the form again: OPTIONS is back before the POST, because a disabled cache doesn't store preflight responses either. Turn the checkbox off.

Action	Requests in Network	Why
Open /filter	GET courses	a simple request: a GET without special headers
Click ❤️	POST like	a simple request: a POST with no body and no special headers
First form submission	OPTIONS enrollments, then POST enrollments	JSON (Content-Type: application/json) takes the request beyond the “simple” kind: the browser asks for permission first
Second form submission	only POST enrollments	the permission is remembered for 600 seconds (Access-Control-Max-Age)
Submission with Disable cache on	OPTIONS and POST every time	the browser cache, and the preflight cache with it, is turned off

Check the server too: open http://localhost:8000/enrollments — the list should contain the records sent from the form (in camelCase: courseId, studentName). Work out for yourself how the like differs from the enrollment — you'll need this answer for the README.

> **The Content-Type header is part of the contract.** If you forget Content-Type: application/json in fetch, the browser sends the body as text/plain: the request becomes “simple,” the preflight disappears, and the backend answers 422 with the text Input should be a valid dictionary or object to extract fields from. Don't mistake this for a server problem: it never received JSON.

> **Check yourself:** the like is saved on the server and survives a reload; the form returns an enrollment number, and for a one-letter name shows the server's message; the preflight is visible for the form and absent for the like. Make a commit.

10. Breaking CORS on Purpose and Finding the Cause

In real life, CORS doesn't break on demand: you just see a red message in the console and don't know where to start. Today you'll practice on controlled breakages: you break exactly one thing, look at how it appears, find the cause using the algorithm, and fix it. The algorithm is from section 7 of the lecture; keep it in front of you:

1. Network: is there a row for your request? No — the request was never sent (a typo in the address, an environment variable that isn't set).
2. Status. 2xx — the server is happy, look for the problem in the data. 4xx and 5xx — read the response body and the backend terminal. CORS error — the browser hid the response, or there was no connection.
3. Response Headers: is there an access-control-allow-origin header, and does its value match the page's address character for character — down to the port and with no trailing slash.
4. Is there an OPTIONS before the request? Check its status: 400 — the origin, method, or header isn't allowed; 405 — CORS isn't connected on the server at all.
5. The FastAPI terminal: a Python traceback means the code needs fixing, not CORS.

> **Commit first.** Before the experiments, commit both projects in their working state. You'll roll back each breakage with git restore main.py (in the frontend, git restore next.config.ts), and it will restore exactly the last commit. If there's no commit with a working backend, the rollback will bring back an old version without CORS.

How to run an experiment

Break one thing at a time and act the same way every time:

1. Introduce the breakage in main.py and save the file. Wait for the line Application startup complete in the backend terminal: the server restarts by itself (likes and enrollments get reset in the process).
2. Open DevTools (Console and Network), turn on Disable cache, and reload the page.
3. Do three things: open /filter; click ❤️ on a course page; submit the enrollment form with a proper name. While you're at it, open /courses too: does the page whose request is sent by the Next.js server work?
4. Write in the journal: what the Console says (the beginning of the message), which rows appeared in Network and with which statuses, which header is missing, and at which step of the algorithm you found the cause.
5. Restore the working code (git restore main.py), make sure everything works again, and only then break the next thing.

Keep the journal in the README.md of the course-catalog project using this template — one entry per breakage:

```
README.md — journal entry template
## Breakage journal

### Breakage 1: <what you broke>
- What I did: <the line of code before and after>
- Console: <the beginning of the message>
- Network: <requests and their statuses>
- Cause: <one sentence>
- How I found it: <the algorithm step>
```

CORS breakages (required)

#	What we break	What we do in the browser
1	Remove CORS entirely: comment out the whole app.add_middleware(...) block	the three actions from the protocol; note separately how /courses behaves
2	A trailing slash: allow_origins=["http://localhost:3000/"]	the same three actions
3	Remove allow_methods: delete the allow_methods=["*"] line entirely	the same three actions: what works and what doesn't?
4	A different origin: main.py is fine, but the frontend is opened at http://127.0.0.1:3000/filter	open /filter, then a course page; a Next.js setting is required (below)

> **Breakage 4 needs one line in the config.** Starting with Next.js 16, the dev server doesn't serve a page opened from a foreign host until that host is listed in allowedDevOrigins. Without it, the page at 127.0.0.1 hangs on “Loading…” and you won't see CORS. Add the line allowedDevOrigins: ["127.0.0.1"] to the nextConfig object in next.config.ts, restart npm run dev, and delete the line after the experiment.

Check your journal

Do breakages 1–4, fill in the journal, and only then compare it with this table. The message texts are given for Chrome.

#	What you should have seen	The cause and how to tell it apart
1	Console on /filter: “Access to fetch at 'http://localhost:8000/courses' from origin 'http://localhost:3000' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource”. In Network the courses request is red (CORS error), even though the backend log says 200 OK. Like: the same message, but the backend log shows POST …/like 200 OK, and the like count on the server went up. Form: OPTIONS enrollments → 405, the POST is not sent. The /courses page works.	There's no access-control-allow-origin in the responses: CORS isn't connected. Two conclusions: the server executes the request even when the browser blocks it; Server Component requests aren't affected by CORS. The difference from #2 — the preflight gets 405, not 400.
2	Console on /filter is the same as in #1. Form: OPTIONS enrollments → 400, the POST is not sent; in Console “Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource”.	The value "http://localhost:3000/" with a slash matches no origin, so permission isn't granted to anyone. Fixed by removing the slash.
3	/filter and ❤️ work. Form: OPTIONS enrollments → 400, the POST is not sent; in Console “Response to preflight request doesn't pass access control check: It does not have HTTP ok status”. The response to OPTIONS has access-control-allow-origin, and access-control-allow-methods: GET.	Only GET is allowed by default, and the POST method is rejected. The access-control-allow-origin header is in place — so what needs fixing is the methods, not the origin.
4	Console: “Access to fetch at 'http://localhost:8000/courses' from origin 'http://127.0.0.1:3000' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present…”. In Network courses is red, the backend log says 200 OK.	To the browser, localhost and 127.0.0.1 are different hosts, and so different origins; only http://localhost:3000 is allowed. Fixed by opening the page at the address from the list, or by adding a second address to allow_origins.

A trick: the preflight by hand

DevTools doesn't always show the body of a preflight response, and that's exactly where the reason for the refusal is named. You can reproduce the preflight from the terminal: substitute your route's address and the page's origin. The command must be typed on one line (in PowerShell — curl.exe):

```
terminal — preflight by hand
curl -i -X OPTIONS http://localhost:8000/enrollments -H "Origin: http://localhost:3000" -H
"Access-Control-Request-Method: POST" -H "Access-Control-Request-Headers: content-type"
```

With a correct setup the response looks like this (other headers omitted, the order of lines may differ; the exact list of methods depends on the Starlette version — what matters is that POST is in it):

```
expected output — preflight allowed
HTTP/1.1 200 OK
access-control-allow-origin: http://localhost:3000
access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT, ...
access-control-allow-headers: content-type
access-control-max-age: 600

OK
```

If something isn't allowed, the server answers 400 and explains in a short text exactly what. Decoder:

Preflight response	What it means
200 OK and access-control-allow-* lines	the origin, method, and headers are allowed — the preflight passes
400 Bad Request, body Disallowed CORS origin, no access-control-allow-origin line	the page's origin isn't in allow_origins: a trailing slash, a foreign host, or a different port
400 Bad Request, body Disallowed CORS method	the requested method isn't allowed: allow_methods isn't set or doesn't include POST (the response then has access-control-allow-methods: GET)
400 Bad Request, body Disallowed CORS headers	the browser asks for a header that isn't in allow_headers
405 Method Not Allowed	CORSMiddleware isn't connected: OPTIONS went to an ordinary route, which doesn't support it

The breakage that pretends to be CORS (required)

Now the trickiest breakage — one where CORS has nothing to do with it. Restore the working main.py and, in the list_courses function, make a typo in a dictionary key: in the last line of the function, replace the first p["offset"] with p["ofset"].

```
main.py — breakage 5
    return courses[p["ofset"] : p["offset"] + p["limit"]]
```

Save the file and do the three actions plus opening /courses. Here's what you'll see:

- /filter shows “Failed to load courses”, and the Console has a familiar message: “No 'Access-Control-Allow-Origin' header is present on the requested resource”. In Network the courses request is red. But CORS is configured correctly!
- ❤️ and the enrollment form work as before: their routes are untouched.
- /courses shows your error.tsx. That page gets its data from the Next.js server, and CORS has nothing to do with it — so the cause isn't CORS (steps 1 and 5 of the algorithm).
- In the backend terminal — status 500 and a Python traceback; the main thing in it is the last lines:

```
backend terminal — window 1
INFO:     127.0.0.1:35342 - "GET /courses HTTP/1.1" 500 Internal Server Error
ERROR:    Exception in ASGI application
Traceback (most recent call last):
  ...
  File ".../main.py", line 42, in list_courses
    return courses[p["ofset"] : p["offset"] + p["limit"]]
                   ~^^^^^^^^^
KeyError: 'ofset'
```

The explanation: an unhandled exception in a route turns into a 500 response, and that response is produced by a layer outside CORSMiddleware — so it has no access-control-* headers. The browser sees a response without permission and shows the same CORS error (section 4 of the lecture). The lesson: first the response status and the backend terminal, and only then the CORS settings. Put p["offset"] back and make sure /filter works again.

Three more breakages without CORS (optional)

These breakages have nothing to do with CORS, but they look similar: there's an error, and the cause isn't obvious.

What to do	What you'll see	Takeaway
In models.py, remove the alias_generator=to_camel line from ConfigDict	There are no errors anywhere, but on /filter the “elective” mark is gone, and the form answers “The server rejected the data: Field required”	Contract drift shows up silently. Look for it by comparing the JSON on the Network tab with the Course type
In .env.local, comment out NEXT_PUBLIC_API_URL and restart the dev server	The request goes to http://localhost:3000/undefined/courses and gets 404. In Console only “Failed to load resource: the server responded with a status of 404 (Not Found)”, no mention of CORS	The address became relative, that is, the same origin. The strange Request URL is visible in the first steps of the algorithm
Stop the backend	In Console “Failed to load resource: net::ERR_CONNECTION_REFUSED”; /courses shows error.tsx	There's no connection — CORS has nothing to do with it (step 2 of the algorithm)

Cheat sheet: what the message means

What you see	Likely cause	What to check
“No 'Access-Control-Allow-Origin' header is present on the requested resource”	CORS isn't connected; the page's origin isn't in the list; or the server answered 500	allow_origins; the backend terminal; the OPTIONS status
“Response to preflight request doesn't pass access control check: It does not have HTTP ok status”	the preflight got 400: the method or header isn't allowed	allow_methods, allow_headers; the body of the preflight response (curl)
net::ERR_CONNECTION_REFUSED	the backend isn't running, or the address or port is wrong	whether fastapi dev is running; NEXT_PUBLIC_API_URL
the request goes to /undefined/courses	NEXT_PUBLIC_API_URL isn't set	.env.local; restarting the dev server
no errors, but the interface shows undefined or nothing	a contract mismatch: the field names in the JSON and in type Course differ	the JSON on the Network tab and the Course type
npm run build fails: fetch failed … ECONNREFUSED	API requests run during the build	generateStaticParams; cache: "no-store"

> **Check yourself:** the journal has at least five entries (breakages 1–5), each with a quote from the console, the statuses from Network, and the cause, and after all the experiments the code in both projects is back in a working state: git status shows only what you changed on purpose.

11. Final Check and README

Before submitting, go through the whole user path from scratch. Stop both servers and start them again (backend first, then frontend): that resets the like counters and enrollments. Then:

1. open /courses: six cards, ai-integration first;
2. go to /courses/web-security, click ❤️ twice and reload the page: the number went up by 2 and was saved;

3. submit the form with the name A (the server refuses), then with the name Aigerim (“Enrollment number: 1”);
4. open http://localhost:8000/enrollments: Aigerim's record is there;
5. open /filter: the checkbox works, and Network shows a request with an Origin header;
6. stop the backend: /courses shows error.tsx, and npm run build passes; start the backend and click “Try again” — the page comes back without a reload.

README. In course-catalog-api/README.md, add a “What changed” section: the allowed origin and why it's needed, camelCase in JSON, the three new routes with example responses. In course-catalog/README.md, describe: how to run both projects (order: backend, then frontend); which variables are needed in .env.local (names and an example value); a “Breakage journal” section (at least five entries following the template from section 10); and a “Decisions” section — two or three sentences on why generateStaticParams was removed from the course page and why the like gets by without a preflight while the enrollment doesn't.

Acceptance Checklist

Before you submit, run through this list — it's exactly what will be checked:

- ☐ fastapi dev main.py starts without errors; /docs shows six routes: GET /, GET /courses, GET /courses/{course_id}, POST /courses/{course_id}/like, POST /enrollments, GET /enrollments;
- ☐ the response to a request with Origin: http://localhost:3000 contains access-control-allow-origin: http://localhost:3000, and the response to a request with a foreign origin doesn't; the preflight on /enrollments answers 200;
- ☐ JSON arrives in camelCase (isElective), and /courses?is_elective=true still returns exactly 2 courses; the models inherit from CamelModel, and validate_by_name is on;
- ☐ POST /enrollments: a valid body — 201 and an id, a one-letter name — 422, an unknown course — 404; POST /courses/nope/like — 404;
- ☐ .env.local is created and didn't end up in git, and the variables are described in the README;
- ☐ /courses and /courses/[id] get their data from the backend (GET requests are visible in the backend terminal); lib/courses.ts has cache: "no-store", the res.ok check, and handles 404 as undefined;
- ☐ app/courses/[id]/page.tsx has no generateStaticParams, and npm run build passes with the backend off;
- ☐ loading.tsx, not-found.tsx, and error.tsx trigger as in the table in section 7; the “Try again” button (retry) brings the page back without a manual reload;
- ☐ on /filter, the “Electives only” checkbox requests ?is_elective=true straight from the browser, and the fetcher checks res.ok;
- ☐ the like is saved on the server (the number doesn't reset after a reload), and the enrollment form shows the enrollment number and, on a 422 response, the server's message;
- ☐ Network shows a preflight (OPTIONS) for the enrollment form and none for the like, and you can explain why;
- ☐ the README has a journal of at least five breakages (four from section 10 and the Python-error one): a console quote, statuses from Network, the cause you found;
- ☐ both repositories are on GitHub: course-catalog-api has at least 3 meaningful commits made today, course-catalog at least 5.

Grading Rubric

Criterion	Points
CORS in the backend: CORSMiddleware with an exact origin, the preflight answers 200, a foreign origin gets no permission	1
The contract: CamelModel (alias_generator and validate_by_name), camelCase responses, query parameters not broken	1
POST routes: the like and the enrollment work per the table in section 4 (200, 201, 404, 422)	1
Frontend: .env.local, getCourses and getCourse through the API (cache: "no-store", res.ok, 404 as undefined), generateStaticParams removed, the build passes without the backend	2
loading.tsx, not-found.tsx, and error.tsx checked with a real backend; error.tsx uses retry	1
Client Components: ElectiveFilter with SWR, LikeButton, and EnrollForm work with the backend; the preflight is seen and explained	2
The breakage journal: at least five entries, causes found by the algorithm	1
Two GitHub repositories with meaningful commits and READMEs	1

Total: 10 points. Adjust the scale to your own grading system as needed.

Common Mistakes

A CORS error in the console, but CORS is configured correctly: almost always it isn't CORS. If the backend answered 500 (an unhandled Python exception), the response is produced by a layer outside CORSMiddleware, it carries no access-control-* headers, and the browser shows the same CORS error. Open the terminal with fastapi dev: the 500 status and the traceback will be there. A hint: if even /courses (which the Next.js server requests) shows error.tsx, it's definitely not CORS.

A trailing slash in allow_origins: the value "http://localhost:3000/" matches no origin: all requests from your page are blocked, and the preflight gets 400 (Disallowed CORS origin). The list holds an origin — scheme, host, and port, with no path and no slash.

localhost and 127.0.0.1 are different origins: if you open the frontend as http://127.0.0.1:3000, you'll get a CORS block even though it's the same computer. The same happens if Next.js took port 3001 because 3000 was busy: the origin became http://localhost:3001. Open the page at an address from allow_origins and stop the extra process on port 3000. (In dev mode, 127.0.0.1 also requires allowedDevOrigins — section 10.)

The request goes to /undefined/courses: the NEXT_PUBLIC_API_URL variable isn't set (a typo in the name, the file isn't in the project root) or the dev server wasn't restarted after the file was created. Network shows a request to http://localhost:3000/undefined/courses with a 404 response, and the console doesn't mention CORS: the address turned out to be relative, that is, the same origin.

API_URL is not set, or process.env.API_URL is undefined in the browser: variables without the NEXT_PUBLIC_ prefix are visible only to server code. In a Client Component use NEXT_PUBLIC_API_URL, and import nothing from lib/courses.ts except types (import type): the module itself is meant for the server and will fail on the API_URL check in the browser.

The page shows stale data, or the build fails: server-side API requests run with cache: "no-store". Without it, Next.js may run the request once during next build and “freeze” the result (likes on the page don't update) or fail if the backend isn't running at that moment: fetch failed … ECONNREFUSED. Another source of requests during the build is generateStaticParams (section 6).

> **Electives vanished or the interface shows undefined:** this is contract drift: there are no errors, but the names in the JSON and in type Course have diverged. Compare the JSON on the Network tab with the Course type. If you changed models.py, check that the model inherits CamelModel and that ConfigDict still has alias_generator and validate_by_name=True.
>
> **The enrollment returns 422 "Input should be a valid dictionary or object…":** the Content-Type: application/json header was forgotten in fetch. The body went out as text/plain, and FastAPI didn't parse it as JSON. Such a request also counts as simple and skips the preflight, so the error is easy to mistake for a server problem.
>
> **Likes and enrollments vanished:** the backend keeps them in memory and resets them on every restart, and fastapi dev restarts on every file save. That's expected; persistent storage arrives with PostgreSQL (weeks 6–7).
>
> **error.tsx: retry is undefined, or the data didn't refresh after clicking:** the retry function is available in Next.js 16.3 and newer (in 16.2 it was called unstable_retry); npm ls next shows the version. The familiar reset only re-renders the component and doesn't request the data again.

Bonus Tasks (optional)

If the core part is done early — pick any, no limit on how many:

1. Rewrites instead of CORS (section 6 of the lecture). In next.config.ts add a rewrites rule: source "/api/:path*", destination "http://localhost:8000/:path*". In ElectiveFilter, change the request address to the relative /api/courses and comment out CORSMiddleware in the backend. On the Network tab, make sure the filter still works: the browser calls localhost:3000, and Next.js itself forwards the request to FastAPI. Leave the like and the form on direct addresses and explain why they're blocked again.
2. Types from OpenAPI. In the frontend folder run npx openapi-typescript http://localhost:8000/openapi.json -o lib/schema.d.ts and replace the hand-written type Course with components["schemas"]["Course"] from the generated file. Then rename any field in models.py, regenerate the types, and see how TypeScript underlines all the old usages (a message like Property … does not exist on type … Did you mean …).
3. The origin list from an environment variable. Implement the option from section 4 of the lecture: allow_origins=os.getenv("CORS_ORIGINS", "http://localhost:3000").split(","). Start the backend with CORS_ORIGINS="http://localhost:3000,http://127.0.0.1:3000" (in PowerShell: first $env:CORS_ORIGINS="…", then fastapi dev main.py) and check with curl that both origins get permission and a foreign one doesn't.
4. Frozen data. Temporarily remove cache: "no-store" from getCourses, run npm run build and npm run start (with the backend running). In the build table /courses will be marked ○. Click a few likes on a course page and compare the number on /courses with the response from http://localhost:8000/courses: the page shows the number as of the build. Put cache: "no-store" back. Why does /courses/[id] stay *ƒ* and show the current number?
5. Explicit lists instead of asterisks. Replace allow_methods=["*"] with ["GET", "POST"] and allow_headers=["*"] with ["Content-Type"]. The app should work as before, while a preflight with the DELETE method or an Authorization header now gets 400 (Disallowed CORS method, Disallowed CORS headers). Explain in one sentence why an explicit list is safer.

How to Submit

Two repositories are submitted: course-catalog-api (the backend) and course-catalog (the frontend) — the same ones as in Labs 4 and 1. You don't need to create new repositories: push today's changes to each.

Commit as you go, not in one chunk at the end. Naming guideline. Backend: feat: enable CORS for the frontend origin → feat: camelCase aliases via CamelModel → feat: like and enrollment routes. Frontend: feat: getCourses and getCourse call the API → fix: remove generateStaticParams → feat: error.tsx with retry → feat: ElectiveFilter with SWR → feat: LikeButton and EnrollForm → docs: breakage journal. The required minimum is 3 commits in the backend and 5 in the frontend.

Send the links to both repositories however your instructor specifies (LMS / course chat).

Deadline: before Lecture 6 begins, unless your instructor states otherwise.